Copenhagen Cupcakes

Projektrapport – webshop til bestilling af cupcakes

Dato: 5. november 2026

Deltager: Mahdi Al-Dailemi (navn skal bekræftes) | Studie-e-mail: ikke oplyst | GitHub: Mahdiald

Klasse: Datamatiker, 2. semester, efterår 2026 (holdbetegnelse ikke oplyst)

GitHub-repository: https://github.com/Mahdiald/Cupcake (verificeret repo; endeligt commit skal kontrolleres)

Illustrativ video: demo/cupcake_demo_SIMULERET.mp4 (simulation med eksempeldata; ikke en reelt optaget brugertest)

Status: Rapportudkast baseret på tilgængelig kildekode og dokumenterede opstartslogs. Fuld test og brugeraccept er ikke dokumenteret.

Indholdsfortegnelse

1. Indledning og baggrund

2. Teknologivalg

3. Krav og acceptkriterier

4. Domænemodel og ER-diagram

5. Mockups og navigation

6. Implementering

7. Test og kvalitet

8. Implementeringsstatus og afgrænsninger

9. Proces og refleksion

10. Aflevering og bilag

Indholdsoversigten er en simpel oversigt. Opdatér eventuelle sidetal i Word inden endelig aflevering.

1. Indledning og baggrund

Dette projekt handler om at udvikle en lokal webbaseret bestillingsløsning for den fiktive kunde Copenhagen Cupcakes. Kunden skal kunne sammensætte cupcakes med valgfri bund og topping, lægge dem i en indkøbskurv og betale med en intern kontosaldo, så en ordre kan hentes i butikken. Løsningen er udviklet som undervisningsprojekt på datamatikeruddannelsens andet semester.

Målgruppen for rapporten er en anden datamatikerstuderende på samme niveau. Rapporten beskriver de valg, der kan observeres i det foreliggende projektmateriale, og skelner mellem implementeret kode og funktioner, der endnu ikke er afprøvet i en afsluttende test.

Kundens væsentlige behov er konto, login, bestilling, overblik over pris, ordrehistorik og administratorens overblik. Webshoppen er beregnet til at køre lokalt på en bærbar computer, ikke som en internet-hostet produktionsløsning.

2. Teknologivalg

Teknologi	Anvendelse i projektet
Java (projektet compiler mod version 17)	Domæneobjekter, HTTP-ruter og databaseoperationer
IntelliJ IDEA (brugt lokalt: 2025.3.2)	Udviklingsmiljø; versionsnummer ses i brugerens kørselslog
Javalin 6.7.0	Webserver og routing på port 7070
Thymeleaf 3.1.3	Server-renderede HTML-skabeloner

PostgreSQL 17 via Docker	Relationel database på port 5432
JDBC / PostgreSQL-driver 42.7.7	SQL-kald via PreparedStatement
BCrypt jBCrypt 0.4	Hashing og kontrol af adgangskoder
Maven	Dependency- og buildstyring
HTML og CSS	Frontend og responsiv styling

Det foreliggende projekt har samlet applikationslogikken i App.java. Det er enkelt for en lille prototype, men en senere version kunne opdeles i controllers, services, repositories og modeller.

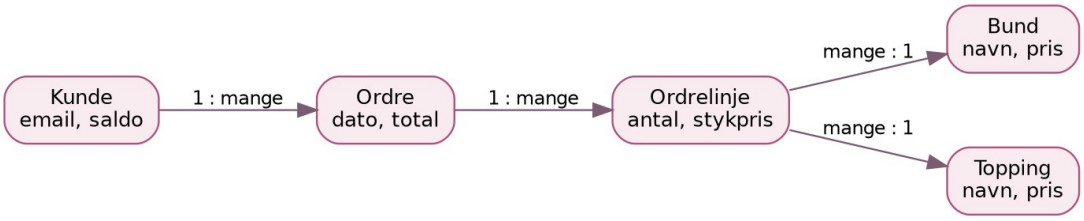
3. Krav og acceptkriterier

Opgavebeskrivelsen angiver otte user stories, hvor de første seks er minimum. Her omsættes kravene til observerbare acceptkriterier. Status er baseret på kodegennemgang, ikke en fuld bestået testkørsel.

Krav	Acceptkriterium	Status
US1	Kunde vælger bund, topping og antal; betaler; ordre gemmes	Kode findes; runtime-test mangler
US2	Kunde registreres med email, password hash og kan genlogge ind	Kode findes; runtime-test mangler
US3	Kurv viser linjer og samlet pris	Kode findes; runtime-test mangler
US4	Login kræver email/kodeord; email vises på siderne	Kode findes; runtime-test mangler
US5	Admin kan se alle ordrer med kunde, beløb og dato	Kode findes; runtime-test mangler
US6	Admin kan se kunder og klikke videre til kundens ordrer	Udvidet i denne pakke; test mangler
US7	Kunde kan fjerne en valgt kurvelinje	Kode findes; runtime-test mangler
US8	Admin kan slette en ordre	Kode findes; ingen automatisk refundering

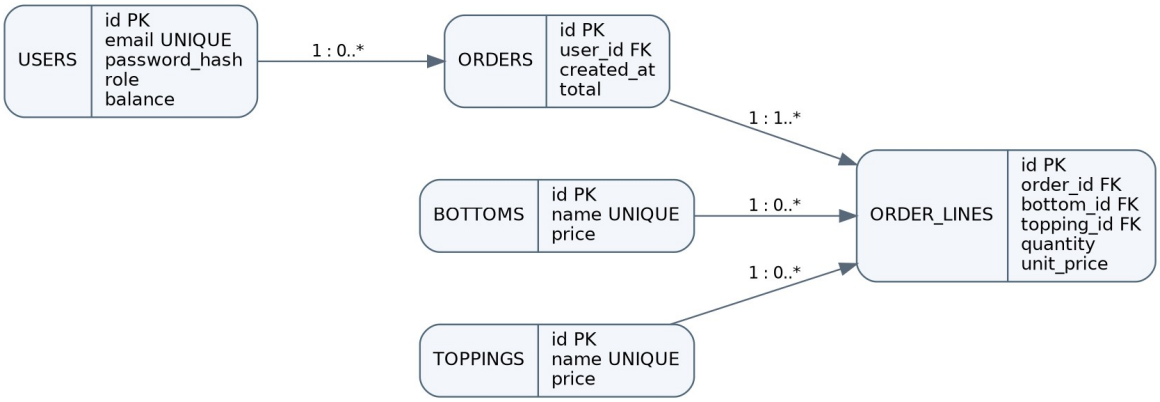
Ikke-funktionelle krav: pengesaldo kan opdateres direkte via PostgreSQL/pgAdmin; kunde- og ordredata gemmes i database; strukturen er designet til 3NF; koden skal deles på GitHub; teknologistakken følger opgaven; mobilvisning skal helst fungere. Sidstnævnte er ikke verificeret på en fysisk telefon.

4. Domænemodel og ER-diagram



Figur: Domænemodel med centrale begreber og relationer.

Kunden kan afgive nul eller mange ordrer. En ordre består af en eller flere ordrelinjer. Hver ordrelinje refererer til præcis én bund og én topping, mens den samme bund eller topping kan bruges i mange ordrelinjer. Dette afspejler de centrale begreber i problemområdet.



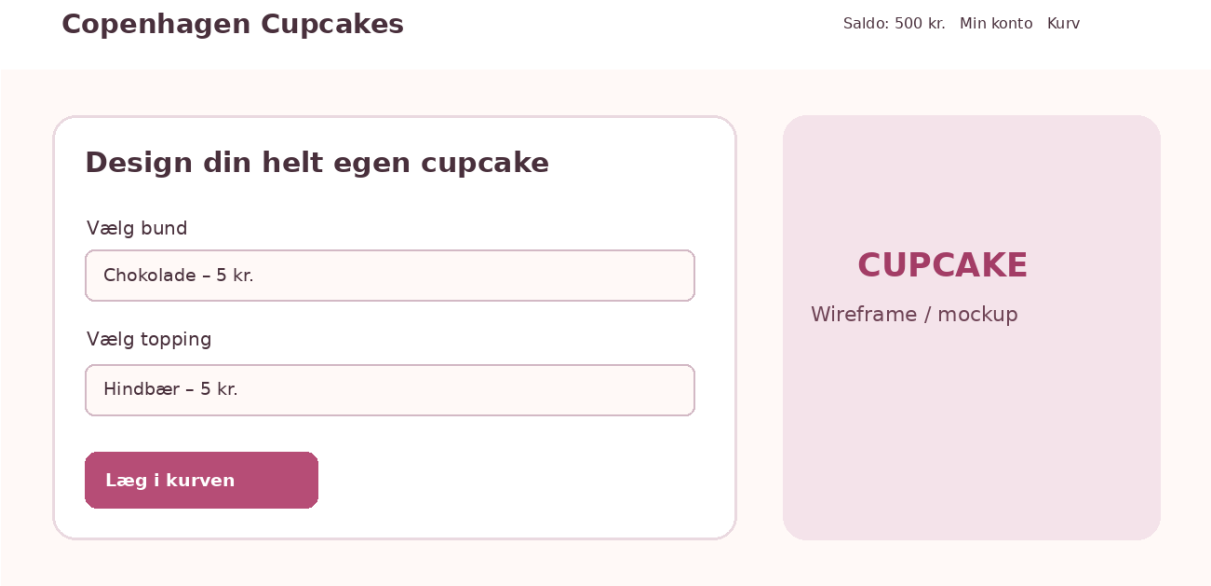
Figur: ER-diagram over de fem databaserelationer.

Databasen består af users, orders, order_lines, bottoms og toppings. Primærnøgler er automatisk genererede ID'er. users.email og produkternes navne er unikke. Fremmednøgler knytter en ordre til kunden, og en ordrelinje til ordre og valgte produkter. ON DELETE CASCADE mellem orders og order_lines sikrer, at linjerne slettes sammen med ordren.

Normalisering: Brugere, ordrer og produktvarianter ligger i separate tabeller, så de ikke gentages i hver ordrelinje. Hver tabel har en entydig nøgle, og ikke-nøglefelter beskriver den pågældende entitet. Ordrelinjens unit_price lagres bevidst som historisk pris på købstidspunktet. Total på ordren lagres også, selvom den kan beregnes fra linjerne; det er et pragmatisk valg, som kræver korrekt transaktionslogik for at undgå inkonsistens.

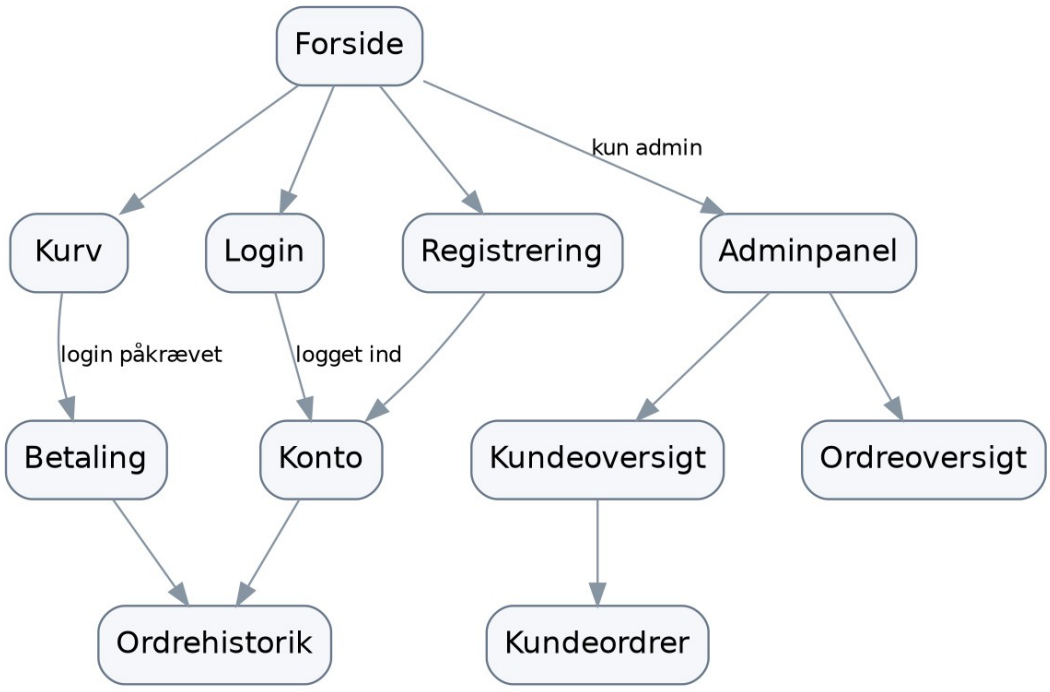
Der anvendes 1-til-mange-relationer frem for mange-til-mange-koblingstabeller, fordi en ordrelinje har præcis en valgt bund og topping. En særskilt eksplicit mange-til-mange-relation er derfor ikke nødvendig i den nuværende model.

5. Mockups og navigation



Figur: Egen lavdetalje-mockup baseret på projektets formularer.

Mockuppen viser idéen til en enkel forside med formular, saldo og navigation. Den er fremstillet som en dokumentationsskitse, ikke som et faktisk skærmbillede af en testet version. Indsæt gerne ægte skærmbilleder fra din lokale browser før aflevering.



Figur: Navigationsoversigt, herunder sider med login- og adminadgang.

Gæster kan se forsiden, tilføje til kurv og besøge login/registrering. Betaling og kontoside kræver en aktiv login-session. Administratorfunktionerne er adgangsbegrænsede via rollen ADMIN. Topmenuen viser email, saldo, kurv og relevante links afhængig af brugerstatus.

6. Implementering

6.1 HTTP-ruter og sessioner

Javalin håndterer GET- og POST-ruter. `uid`, `email` og `role` gemmes som sessionsattributter, og indkøbskurven gemmes i sessionen som liste af ordrelinjer. Ved login og oprettelse skiftes session-ID. Med Thymeleaf renderes serverdata direkte ind i HTML.

6.2 Køb og transaktioner

Ved checkout beregnes totalbeløbet ud fra linjernes antal og enhedspris. SQL-kommandoen trækker kun saldo, hvis brugeren har penge nok (`balance >= total`). Ordre og ordrelinjer indsættes i samme databasetransaktion. Ved exception udføres rollback, og ved succes tømmes kurven. Det beskytter mod, at betaling og ordre bliver delvist gemt.

6.3 Sikkerhed og validering

Login anvender BCrypt til kodeordshash, og SQL-opslag bruger parametre frem for direkte brugerskabte SQL-strenge. Formularen validerer bl.a. antal (1–100), email i enkel form og mindst otte tegn i kodeord. Adminruterne kræver rollen ADMIN. Der mangler dog blandt andet CSRF-beskyttelse, stærkere inputvalidering og produktionssikker konfiguration.

6.4 Saldo, kontoside og administration

Saldo hentes fra databasen ved sidevisning for en logget ind bruger. `/account` viser egen email, saldo og egne ordrer ved filtrering på user_id. Administrator kan se alle ordrer og kunder, vælge en kunde for at se kundens ordrer og slette en ordre. Kodeændringen til den detaljerede kundeordrevisning er inkluderet i afleveringspakken.

7. Test og kvalitet

Testgrundlaget skal være ærligt: der foreligger en lokal opstartslog, der viser Javalin på localhost:7070, en Docker-besked om at PostgreSQL accepterer forbindelser, samt et browserbillede med forventet afvisning af køb ved manglende saldo. Disse observationer udgør ikke et fuldt accepttestforløb.

Testtype	Dækning	Evidens/status
Opstartstest	Javalin-server og databaseforbindelse	Observeret i dialog/log
Negativ checkout	Afvisning ved for lav saldo	Fejlmeddelelse observeret
Funktionel test US1–US8	Flows med konti, kurv, betaling og admin	Skal køres på lokal maskine

Integrationstest	Saldo- og ordrekonsistens i PostgreSQL	Planlagt, ikke kørt
Brugertest / UAT	Bruger gennemfører køb uden hjælp	Ikke dokumenteret
Automatiske tests	JUnit/testsuite	Ikke til stede i projektudkast



Figur: Faktisk observeret afvisning af betaling ved utilstrækkelig saldo (browserfoto fra udviklingsforløbet).

Se `tests/TESTPLAN.md` for en konkret testliste på 16 cases og SQL til efterkontrol. Opdatér denne tabel med faktiske resultater efter afprøvning.

8. Implementeringsstatus og afgrænsninger

- US1–US5, US7 og US8 har implementerede ruter og skabeloner; endelig accept afventer test.
- US6 er udvidet i denne pakke med navigation til en valgt kundes ordreoversigt; dette er endnu ikke kørt på brugerens PC.
- Indbetaling foregår ved SQL i PostgreSQL, ikke i selve hjemmesiden, hvilket følger opgavekravet.
- Ordresletning giver ikke pengene tilbage; en rigtig refundering kræver en besluttet forretningsregel og en transaktion.
- En betalingsgateway, e-mailkvittering og online hosting er uden for denne prototypes omfang.
- Der foreligger ikke dokumenterede automatiske tests, mobiltest eller registreret UAT.

9. Proces og refleksion

Det dokumenterbare projektforløb er iterativt: først blev en Javalin-prototype oprettet, derefter blev fejlen med databaseforbindelsen identificeret. PostgreSQL blev startet i Docker, og webshoppen kunne efterfølgende nå checkout. En fejlsituation med utilstrækkelig saldo blev observeret. Derefter blev saldo/ordrehistorik tilføjet, og denne afleveringspakke omfatter også en udvidelse af administratorens kundeordreoversigt.

Der foreligger ikke pålidelige oplysninger om gruppens interne roller, møder, brug af GitHub Projects, pull requests eller commitfrekvens. Disse oplysninger må udfyldes af deltagerne og ikke fremstilles som gennemførte aktiviteter. Forslag til refleksion: En kort opdeling i database, backend, frontend og test ville have gjort opgaverne lettere at planlægge; fremover bør tests skrives tidligere og knyttes til hver user story.

10. Aflevering og bilag

Koden, database-scriptet og diagrammerne ligger i projektmappen. Den endelige rapport gemmes som `rapport/rapport.pdf` på GitHub sammen med koden. I bilaget `demo/cupcake_demo_SIMULERET.mp4` findes en simuleret præsentation af brugerflowet (ca. 1 minut og 29 sekunder). Den er tydeligt mærket og kan ikke bruges som evidens for, at løsningen er testet i drift. Kravet om en reel video-demo af det fungerende system skal derfor stadig opfyldes. GitHub-linket afleveres på Moodle; deadline skal kontrolleres i Moodle.

Bilag: `database/schema.sql`; `docs/erd.mmd`; `docs/domaenemodel.mmd`; `docs/navigation.mmd`; `tests/TESTPLAN.md`; `demo/VIDEO_MANUSKRIPT.md`; `demo/cupcake_demo_SIMULERET.mp4`.

Kildegrundlag: skolens ”Cupcake projekt” (4 sider), skolens ”Cupcake rapportskabelon” (5 sider), samt projektets Java-, Thymeleaf- og SQL-filer som gennemgået 5. november 2026.